

Método	Descripción
close()	Finaliza la conexión entre el programa y el archivo en el disco. Es vital usarlo para liberar recursos del sistema operativo y asegurar que todos los datos pendientes se guarden correctamente.
detach()	Separa el búfer subyacente del flujo de datos (TextIO) y lo devuelve. Una vez usado, el objeto original deja de ser utilizable. Se usa en niveles avanzados de manipulación de flujos de datos.
fileno()	Devuelve un número entero conocido como "file descriptor". Es el identificador que usa el sistema operativo (Unix/Windows) para rastrear el archivo abierto a nivel de kernel.
flush()	Fuerza la escritura de los datos que están en el búfer de memoria interna hacia el disco físico. Útil para asegurar que no se pierdan datos si el programa se interrumpe sin haber cerrado el archivo.
isatty()	Comprueba si el archivo está conectado a un dispositivo de terminal (como la consola). Devuelve True si es así y False si es un archivo de texto normal en el disco.
read(n)	Lee y devuelve el contenido del archivo. Si se especifica un número n, lee solo esa cantidad de bytes o caracteres; si no, lee todo el archivo hasta el final (EOF).
readable()	Verifica los permisos de apertura. Devuelve True si el archivo fue abierto en un modo que permite la lectura (como 'r' o 'r+').
readline()	Lee el archivo carácter por carácter hasta encontrar un salto de línea (\n) o llegar al final del archivo. Ideal para procesar archivos grandes línea por línea sin saturar la memoria RAM.
readlines()	Lee todas las líneas del archivo y las organiza dentro de una lista de Python, donde cada elemento es una línea de texto.
seek(offset)	Mueve el cursor de lectura/escritura a una posición específica (byte). Por ejemplo, seek(0) vuelve al inicio del archivo para poder leerlo de nuevo desde el principio.
seekable()	Indica si el archivo permite el acceso aleatorio (moverse a cualquier posición). La mayoría de los archivos en disco lo permiten, pero flujos de datos en red o tuberías (pipes) no siempre.
tell()	Informa la ubicación exacta (en número de bytes) en la que se encuentra el cursor en ese momento. Es como preguntar "¿en qué página del libro me quedé?".
truncate(size)	Recorta el archivo al tamaño indicado en bytes. Si no se especifica un tamaño, lo corta en la posición actual del cursor. Los datos que sobran se eliminan permanentemente.
writable()	Confirma si el archivo fue abierto con permisos de escritura (modos como 'w', 'a' o 'r+'). Si es False, intentar escribir lanzará un error.
write(string)	Inserta una cadena de texto dentro del archivo en la posición actual del cursor. No añade saltos de línea automáticamente; hay que incluirlos manualmente con \n.
writelines(list)	Toma una lista de cadenas de texto y las escribe de forma consecutiva en el archivo. No añade separadores, por lo que cada elemento de la lista debe incluir su propio salto de línea.